

Registers -SRAM Implementation

For this assignment, I implemented the General Purpose Registers (GPR), Program Counter (PC), Instruction Register (IR), 8-bit Adder, and 8-bit SRAM using Digital. I built each major component as its own subcircuit first (GPR.dig, PC.dig, IR.dig, Adder.dig, SRAM.dig), and then connected everything in a top-level circuit (RegisterSRAM).

Keeping each component separate made it easier to debug before integrating the full system.

General Purpose Registers (GPR)

In my GPR design, I created four 8-bit registers (R0–R3).

From the screenshots:

- A 2-bit “pick register” input selects which register is active.
- A decoder is used to generate individual enable signals for R0–R3.
- A MUX selects which register’s output is sent to the GPR output.
- A shared clock drives all four registers.
- A ctrl signal controls when data is written.

This structure separates read and write logic. The decoder handles writing, and the MUX handles reading.

At first, I had issues because the select signal did not match the number of registers, which caused undefined wire errors. After fixing the select width and making sure the decoder and MUX were properly connected, the error was resolved.

Program Counter (PC)

The PC is implemented as an 8-bit register with increment functionality.

Based on my design:

- An 8-bit adder adds the current PC value and a constant (1).
- A MUX selects between the incremented value and a loaded value.
- The PC_INC signal controls increment behavior.
- The PC is connected to the system clock.

Initially, I placed the adder at the top level, but I realized the increment logic belongs logically inside the PC structure since it directly modifies the PC value. This better shows how real CPU hardware works.

The PC output is connected to the SRAM address input, allowing instruction fetch.

Instruction Register (IR)

The IR is an 8-bit register with:

- IR_LD control signal
- Clock input
- Data input connected to SRAM DOUT

In the top-level screenshot, SRAM DOUT connects directly to the IR input. When IR_LD is enabled and the clock pulses, the instruction is stored in the IR.

This matches the instruction fetch stage of a CPU:

PC → SRAM → IR

SRAM Module

The SRAM is configured as:

1. 8-bit data width
2. 4-bit address width (16 addresses)
3. Control signals: CS, WE, OE

In my design:

- PC output connects to SRAM address.
- GPR output connects to SRAM DIN for writing.
- SRAM DOUT connects to IR for instruction loading.

The control signals are exposed at the top level so memory operations can be manually tested.

Top-Level Integration (RegisterSRAM)

The top-level circuit connects:

- PC → SRAM address
- SRAM DOUT → IR
- GPR → SRAM DIN
- Shared clock → PC, GPR, IR
- Control signals → PC_INC, GPR_LD, IR_LD, WE, CS, OE

This top-level structure allows simulation of:

- Instruction fetch
- Memory read/write
- Register loading

Without the top-level circuit, the components would function individually but not as a complete system.

Challenges and Debugging

Some issues I encountered:

- Bit-width mismatches on select lines.
- Undefined wire errors when select signals were not properly driven.
- File reference errors when renaming subcircuits (Digital still looked for old names).
- Confusion about whether the adder belonged inside the PC or at the top level.

Debugging these helped me understand:

- Every wire must have exactly one driver.
- Select signals must match the number of components controlled.
- Integration is as important as individual component design.

Conclusion

Through this assignment, I learned how registers interact with memory and how control signals coordinate data movement in a CPU. Building each module separately and then integrating them at the top level helped me understand the structure of a basic processor.

Even when parts initially did not work correctly, debugging the errors improved my understanding of digital system design and hardware organization. Although I still wasn't able to get everything to work at the top-level circuit. Maybe I should have started with that. The screenshots are below.



